COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	09/08/2026	**DATE SUBMITTED:**	09/08/2026
NAME:	Troy Jerron A. Bautista	**INSTRUCTOR:**	Sir Jimlord Quejado

ACTIVITY NO. 7.1	**Sorting Algorithms Pt1**

1. PROCEDURE OUTPUT

Code + Console Screenshot	**Main Program:**
	6 const int size = 100;
	7 void displayArr(int arr[]);
	8
	9 int main() {
	10 // generate 100 random numbers
	11 int rand_num[size];
	12 srand(time(0));
	13 for (int i = 0; i < size; i++) {
	14 rand_num[i] = rand() % 100;
	15 }
	16 // output 100 random numbers
	17 displayArr(rand_num);
	
	Output:
	31 40 39 22 76 16 72 37 65 65 5 97 4 69 87 65 58 21 20 42 57 58 93 76 83 76 8 50 35 8
	9 51 81 22 40 71 80 73 91 36 23 14 85 92 73 92 32 54 66 98 62 5 93 10 22 61 46 30 18
	50 63 43 27 63 80 46 12 36 13 84 58 30 79 0 16 22 94 27 82 61 49 27 19 47 18 28 67 51
	23 31 30 86 41 67 51 6 29 81 68 49 20

Observations	Based on my observartion, this code generates random numbers from 1 to 100. The srand is used to initializes the seed value of the number, while the rand produces the random number, which are numbers modulo of 100. The displayArr is function that displays all 100 random number in the console, hence getting the output, showing 100 randomly generated numbers.

Table 7-1. Array of Values for Sort Algorithm Testing

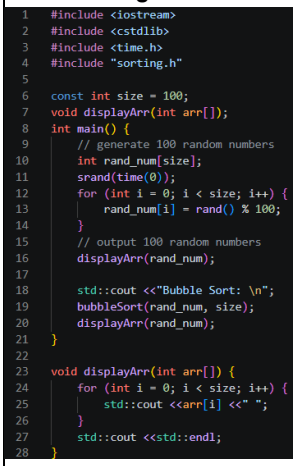
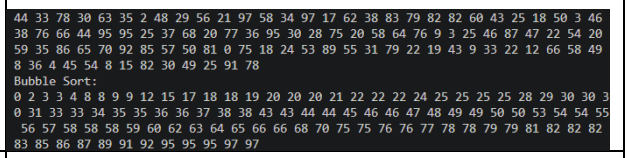
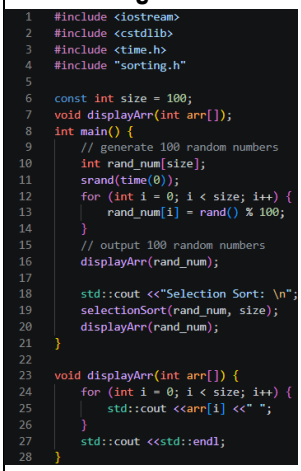
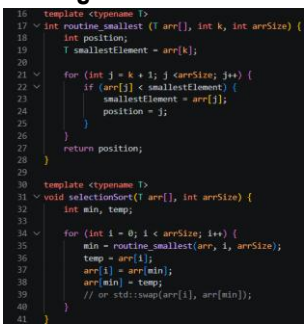
Code + Console Screenshot	**Main Program** **sorting.h**   **Output:**  44 33 78 30 63 35 2 48 29 56 21 97 58 34 97 17 62 38 83 79 82 82 60 43 25 18 50 3 46 38 76 66 44 95 95 25 37 68 20 77 36 95 30 28 75 20 58 64 76 9 3 25 46 87 47 22 54 20 59 35 86 65 70 92 85 57 50 81 0 75 18 24 53 89 55 31 79 22 19 43 9 33 22 12 66 58 49 8 36 4 45 54 8 15 82 30 49 25 91 78 Bubble Sort: 0 2 3 3 4 8 8 9 9 12 15 17 18 18 19 20 20 20 21 22 22 22 24 25 25 25 25 28 29 30 30 30 31 33 33 34 35 35 36 36 37 38 38 43 43 44 44 45 46 46 47 48 49 49 50 50 53 54 54 55 56 57 58 58 58 59 60 62 63 64 65 66 66 68 70 75 75 76 76 77 78 78 79 79 81 82 82 82 83 85 86 87 89 91 92 95 95 95 97 97
Observations	In this observation, in the sorting.h file, it uses a nested loop to determine if arr[i] is greater or less than arr[j], which if arr[i] is greater, then arr[i] and arr[j] would swap, so that largest goes at the end. This sorting algorithm has a time complexity of $O(n^2)$ due to having a nested loop.

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	**Main Program** **sorting. h**  

	Output: 83 42 40 78 66 20 83 89 60 15 47 26 13 82 59 98 66 65 83 31 96 45 30 82 52 51 8 3 58 83 19 63 77 44 48 32 38 3 32 98 84 79 92 19 31 87 44 7 24 6 14 6 1 29 3 43 58 40 54 7 4 70 22 82 0 62 38 96 90 54 54 64 51 65 1 12 26 16 94 21 11 14 14 31 45 86 6 31 28 56 69 39 32 81 99 8 49 41 67 43 72 Insertion Sort: 0 1 1 3 3 3 6 6 6 7 8 8 11 12 13 14 14 14 15 16 19 19 20 21 22 24 26 26 28 29 30 31 3 1 31 31 32 32 32 38 38 39 40 40 41 42 43 43 44 44 45 45 47 48 49 51 51 52 54 54 54 56 58 58 59 60 62 63 64 65 65 66 66 67 69 70 72 74 77 78 79 81 82 82 82 83 83 83 84 86 87 89 90 92 94 96 96 98 98 99
Observations	Based on my observation, this is an output of a selection sort, that has a time complexity of $O(n^2)$. In the sorting.h file, we can see that selection sort has two for loops, one in the routine_smallest, that will be used in the selection sort function, one in the function itself. In this program a similar solution, yet different approached is used to move the smallest numbers at the front of the array.

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	**Main Program**
	`#include <iostream>`
	`#include <cstdlib>`
	`#include <time.h>`
	`#include "sorting.h"`
	
	`const int size = 100;`
	`void displayArr(int arr[]);`
	
	`int main() {`
	`  // generate 100 random numbers`
	`  int rand_num[size];`
	`  srand(time(0));`
	`  for (int i = 0; i < size; i++) {`
	`  rand_num[i] = rand() % 100;`
	`  }`
	`  // output 100 random numbers`
	`  displayArr(rand_num);`
	
	`  std::cout <<"Insertion Sort: \n";`
	`  insertionSort(rand_num, size);`
	`  displayArr(rand_num);`
	`}`
	
	`void displayArr(int arr[]) {`
	`  for (int i = 0; i < size; i++) {`
	`  std::cout <<arr[i] <<" ";`
	`  }`
	`  std::cout <<std::endl;`
	`}`
	
	sorting.h
	`template <typename T>`
	`void insertionSort (T arr[], int arrSize) {`
	`  int k = 1, j;`
	`  T temp;`
	
	`  while (k < arrSize) {`
	`  temp = arr[k];`
	`  j = k - 1;`
	`  while (j >= 0 && temp <= arr[j]) {`
	`  arr[j + 1] = arr[j];`
	`  j--;`
	`  }`
	`  arr[j + 1] = temp;`
	`  k++;`
	`  }`
	`}`
	`#endif`
	
	Output:
	69 22 26 0 41 78 18 9 19 57 77 89 5 17 23 1 51 11 16 31 70 45 15 81 7 10 86 80 81 34
	76 9 17 35 22 3 65 19 78 37 86 14 12 29 93 51 93 77 16 38 58 21 58 25 67 29 21 8 88
	0 23 1 8 17 29 12 71 69 98 49 91 34 40 59 31 18 14 61 84 67 10 87 13 42 11 31 57 9 61
	21 58 97 88 16 8 8 67 33 58 7
	Selection Sort:
	0 1 1 3 5 7 7 8 8 8 8 9 9 9 10 10 11 11 12 12 13 14 14 15 16 16 16 17 17 17 18 18 19
	19 20 21 21 21 22 22 23 23 25 26 29 29 29 31 31 31 33 34 34 35 37 38 40 65 41 42 78
	5 49 51 51 57 57 58 58 58 58 59 61 61 67 67 67 69 69 70 71 76 77 77 78 80 81 81 88 84
	86 86 87 88 89 91 97 93 98 93

Observations	In this observation, this is a insertion sort, which is, in my opinion, the fastest to sort an array. In the sorting.h file, instead of using a for loop, it is more convenient and easier to use a while loop in this sorting algorithm. It also uses a temporary algorithm and has a condition of j >= 0, that avoid the code from output garbage numbers. This has a best-case time complexity of O(n), and the worst-case of $O(n^2)$.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Pseudocode of Algorithm

Main Program

```
START
SET winner to 0
SET max_size to 100
Set A to maax_size – 1
FOR I from 0 to max_size – 1
    A[I] to random(1,5)
ENDFOR
CALL display_votes with A
CALL sortVotes with A and max_size
CALL display_votes with A
FOR each index of 5
    CALL countVotes with max_size, I + 1, and A
    STORE result in vote_count[I]
    SET max to max(vote_count)
    SET winner to I
ENDFOR
    PRINT candidate[winner]
    PRINT max
END
```

Function of display_votes

```
FUNCTION display_votes (votes[])
    FOR I from 0 to max_size - 1
        PRINT votes[I]
    ENDFOR
ENDFUNCTION
```

Sorting.h file

sortVotes function

```
FUNCTION sortVotes (T arr[], int arrSize)
     SET K to 1
     WHILE K < arrSize
         SET temp_var as arr[K]
         SET J as k - 1
         WHILE J >= 0 AND temp_var <= arr[J]
             SET arr[J + I] as arr[J]
             DECREMENT J
         ENDWHILE
         SET arr[J + I] as temp_var
         INCREMENT K
END FUNCTION
```

countVotes function

```
FUNCTION countVotes (int arrSize, T voteKey, T arr[])
    SET I as 0
    SET vote_count as 0
```

```
    WHILE I < arrSize
        IF voteKey == arr[I]
            INCREMENT vote_count
        ENDIF
    ENDWHILE
    PRINT vote_count
    RETURN vote_count
ENDFUNCTION
```

Screenshot of Algorithm Code

Main Program

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include <string>
#include <algorithm>
#include "votingCount.h"

const int max_size = 100;
void display_votes(int votes[]);
std::string candidates[5] = {"Bo Dalton Capistrano",
                             "Cornelius Raymon Agustin",
                             "Deja Jayla Banaga",
                             "Lalla Brielle Yabu",
                             "Franklin Relano Castro"};
int vote_count[5] = {};

int main() {
    int max;
    int winner = 0;
    //generate 100 numbers containing votes
    int A[max_size];
    srand(time(0));
    for (int i = 0; i < max_size; i++) {
        A[i] = rand() % 5 + 1;
    }
    display_votes(A);

    std::cout <<"\nSorting votes..." <<std::endl;
    std::cout <<"\nVotes sorted!" <<std::endl;
    sortVotes(A, max_size);
    display_votes(A);

    std::cout <<"\nFinal Election Votes:" <<std::endl;
    for (int i = 0; i < 5; i++) {
        std::cout <<candidates[i] <<": ";
        vote_count[i] = countVotes(max_size, i + 1, A);
        max = vote_count[0];
        max = std::max(max, vote_count[1]);
        max = std::max(max, vote_count[2]);
        max = std::max(max, vote_count[3]);
        max = std::max(max, vote_count[4]);
        if (max == vote_count[i]) {
            winner = i;
        }
    }
    std::cout <<"\nCandidate Winner: " <<candidates[winner] <<std::endl;
    std::cout <<"Total Votes: " <<max <<" votes";
}

void display_votes (int votes[]) {
    for (int i = 0; i < max_size; i++) {
        std::cout <<votes[i] <<" ";
    }
    std::cout <<std::endl;
```

votingCount.h

```h
#ifndef VOTINGCOUNT_H
#define VOTINGCOUNT_H
#include <iostream>

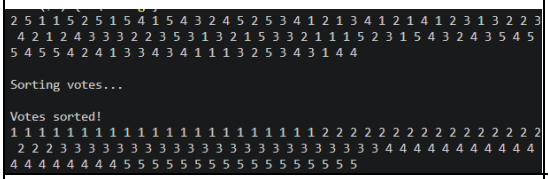
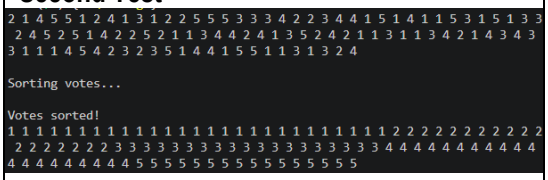
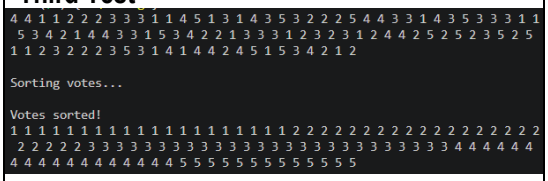
template <typename T>
void sortVotes (T arr[], int arrSize) {
    int k = 1, j;
    T temp_var;

    while (k < arrSize) {
        temp_var = arr[k];
        j = k - 1;
        while (j >= 0 && temp_var <= arr[j]) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = temp_var;
        k++;
    }

}

template <typename T>
int countVotes (int arrSize, T voteKey, T arr[]) {
    int i = 0;
    int vote_count = 0;

    while (i < arrSize) {
        if (voteKey == arr[i]) {
            vote_count++;
        }
        i++;
    }
    std::cout <<vote_count <<" votes" <<std::endl;
    return vote_count;
}
#endif
```

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
First Test 	Candidate 1 = 22 Candidate 2 = 19 Candidate 3 = 23 Candidate 4 = 19 Candidate 5 = 17	Final Election Votes: Bo Dalton Capistrano: 22 votes Cornelius Raymon Agustin: 19 votes Deja Jayla Banaga: 23 votes Lalla Brielle Yabu: 19 votes Franklin Relano Castro: 17 votes
Second Test 	Candidate 1 = 27 Candidate 2 = 18 Candidate 3 = 19 Candidate 4 = 20 Candidate 5 = 16	Final Election Votes: Bo Dalton Capistrano: 27 votes Cornelius Raymon Agustin: 18 votes Deja Jayla Banaga: 19 votes Lalla Brielle Yabu: 20 votes Franklin Relano Castro: 16 votes
Third Test 	Candidate 1 = 20 Candidate 2 = 23 Candidate 3 = 26 Candidate 4 = 18 Candidate 5 = 13	Final Election Votes: Bo Dalton Capistrano: 20 votes Cornelius Raymon Agustin: 23 votes Deja Jayla Banaga: 26 votes Lalla Brielle Yabu: 18 votes Franklin Relano Castro: 13 votes

Question: Was your developed vote counting algorithm effective? Why or why not?
My developed vote counting algorithm was effective because it managed to count all the votes of correctly, with no margin of error. On the three tests I attempted, all the voting counts, were not only accurate, but also total to 100, which is the max size of the array. Furthermore, it also displayed which candidate has the highest vote and the winner of the elections.

Conclusion:
In conclusion, sorting algorithms are ways to sort numbers in order. There are 3 basic types of sorting algorithms and each of them have different ways to sort each number. The bubble sort compares the numbers adjacent to it and determine which one is higher. The selection sort selects the lowest number in the array and moves it to the back of the array, and inserts the value between the two values greater and less than the value inserted.

The procedure show how simple sorting algorithms can be. Based on observations, I can say that bubble sort is the slowest, since it iterates the whole array before repeating it again, without going to back to arrange it properly, while insertion sort is the fastest, since it inserts the value immediately between two numbers. All the worst-case time complexity of all of these sorting algorithms are $O(n^2)$ since they have a nested loop.

The supplementary gave us a scenario to not only sort the votes using sorting algorithms, but also use searching algorithms to add all the instances of the same number, and determine which candidate is the winner. The first few parts of the supplementary are fairly easy as most of these were already made from the procedures and the previous supplementary activities., until to determining which candidate won, which just need extra variables in order to successfully solve the problem

Overall, I can say that I learned a lot in this activity and fairly did good at the coding, since I managed to solve the sorting part of the algorithm, using the codes from the procedure. For now, I don't think I have anything to improve about my coding, since I understand how most these work, and which ones are efficient in certain situations.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT and Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of Activity, Pseudocode Assistance, and Grammar Correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **By using ChatGPT, I can confirm that I only used it for the following, while still the work on my own, and only used it as a helping hand**
By using Grammarly, I was able to correct any mistakes I made in my grammar and make write my conclusion cleanly and clearly.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.